Writing a BenchPanel plugin with Claude

How to use **Claude Code** (in your terminal) or **Claude.ai** (in the browser) to write a plugin for an instrument BenchPanel doesn't support yet. Claude can get you there fast; it can also be confidently wrong about an instrument it has never seen. This guide is about getting the first without the second.

Every rule here comes from something that actually went wrong while the plugins in BenchPanel's `catalog/` were written: an E3641A supply, a 34401A meter, an 8648A signal generator, a 33522A function generator and a 53181A counter. The repository has three companion documents: `PLUGINS.md` (the how-to), `PLUGIN_STANDARD.md` (rules S0–S16 a plugin must meet) and `CLAUDE.md` (these rules, read by Claude Code automatically).

Which one to use

	CLAUDE CODE	CLAUDE.AI
Works in	your terminal, inside the BenchPanel folder	a browser chat
Reads the repo	yes, by itself	only the files you upload
Runs the checker	yes, and fixes what fails	no: you run it and paste the output back
Talks to your instrument	through scripts it writes and runs; you approve each one	no: you run its scripts and paste the results back
Best for	the whole job, start to finish	when you can't run Claude Code on the bench machine

Both end in the same place: a plugin folder that passes `tools/check_plugin.py` and has been tried on the real instrument.

Before you start

- **The right programming manual, for your exact model.** This matters more than anything else here. Sister instruments look alike and have different command sets. The 53181A plugin was nearly written from the 53131A/132A guide, because that was the file labelled "53131/53181". Check the title page for the model number and the serial-number prefix it covers.
- **The instrument**, powered, connected, and with a known GPIB address, IP address or VISA resource string.
- **BenchPanel running**, with `pip install -r requirements.txt` done.
- **Its current settings**, written down or photographed, so you can put them back.

The rules to hold Claude to

With Claude Code, the repo's `CLAUDE.md` gives it these already. With Claude.ai, the starter prompt carries them.

1 **Every SCPI command comes from the manual, with a page number.**
Not from memory, and not from a similar model's manual. If the manual doesn't say, the answer is "the manual doesn't say".

2 **Check the page citations.**
Claude will sometimes quote the right text and attach a page from a different document. In the 53181A driver, two of six page references came from the wrong guide.

3 **Nothing that can block the bus.**
A counter with no signal, or a meter waiting on a trigger, can hold the bus until the measurement completes. The 53181A's `READ?` with no signal wedged the connection until a device clear. The driver must start a measurement, ask whether it has finished using something that never blocks, and only then fetch.

4 **Read-only first.**
Before any driver exists, Claude writes a probe that only queries settings, never measures, and hands the front panel back. You see what your instrument actually says.

5 **Outputs need your say-so.**
Nothing that sources power or signal is switched on without your agreement, and output-on actions are declared `"confirm"` in the manifest.

6 **Record, then restore.**
Anything the driver changes on connect (run mode, trigger source) is recorded first and put back in `on_release()`.

7 **Passing the checker is necessary, not sufficient.**
It proves the plugin is safe and well-formed. Only the instrument proves the commands are right.

8 **Evidence, not claims.**
"It works" means readings compared with a known source, the error stated, and the instrument's error queue empty.

With Claude Code

Open a terminal in the BenchPanel folder and start `claude`. It reads `CLAUDE.md` automatically. Each step below is a prompt you can paste, with your details filled in.

1 Hand it the manual and a plugin to copy

PROMPT

```
I want a BenchPanel plugin for the [MAKE MODEL]. The programming
guide is at ~/manuals/[FILE].pdf. First confirm from its title page
that it covers this model and my serial prefix [PREFIX]. Then read
PLUGIN_STANDARD.md, PLUGINS.md, and the catalog plugin most like this
instrument, and tell me which one you'll follow and why. Don't write
any files yet.
```

Closest examples: `catalog/34401a` (no outputs), `catalog/e3641a` (a supply or anything with an output), `catalog/33522a` (two channels, saved states), `catalog/53181a` (measurements that can wait forever).

2 Read-only probe

PROMPT

```
Write a read-only probe script for my [MODEL] at [ADDRESS]. It should only send queries (no measurements, no READ?/MEAS?/FETC?, nothing that changes a setting), print each reply with its timing, and hand the front panel back at the end. Show me every command it will send before you run it.
```

Keep the output: it's the instrument's real starting state, and what you'll restore later.

3 The measurement design, before any code

PROMPT

```
From the manual, how do I start a measurement, find out it has finished without blocking, and fetch it? Quote the manual and give page numbers. What happens if there's no signal or no trigger?
```

If the answer doesn't name a register or event that never blocks, push back. This is where most counter and meter drivers go wrong.

4 Write it, and make it pass

PROMPT

```
Write the plugin in catalog/[ID]/: manifest.json, driver.py, card.html, card.js, README.md. Every command cited to a page. Then run python3 tools/check_plugin.py catalog/[ID] and fix it until it passes with no MUST failures. Show me the final checker output.
```

5 Try it with no hardware

PROMPT

```
Run the panel with a scratch site file and plugins directory, start
tools/sim_scpi.py --model [MODEL], add it as a VISA resource, and
check the card loads and the power button works.
```

The simulator proves the wiring, not the commands.

6 Test on the real instrument

PROMPT

```
Plan a test on the real instrument. Tell me how to wire it, what
known source you'll compare against, what you expect to see, and how
you'll restore the settings from the probe. Don't switch any output
on until I say so.
```

Wire it as asked, say go, and look for:

- readings compared with a known source, with the error stated;
- the instrument's error queue empty;
- the unhappy case tried: no signal, over-range, instrument switched off;
- a second instrument on the same bus still working throughout;
- afterwards, the settings read back from the instrument itself, matching the probe.

7 Finish and install

PROMPT

```
Record what we tested in the manifest's tested_on and in README.md,
bump the version, and run the checker and tests/test_conformance.py
again.
```

Then install it from the dashboard: **Instruments** → **Get plugins**. It's in the catalog list, or zip the folder and use **Check this .zip**.

With Claude.ai

Claude.ai can't run anything on your machine, so you are its hands. It writes, you run, you paste back.

1 Set up a chat, or better a Project

A Project keeps the files with you across conversations. Upload the programming manual PDF, `PLUGIN_STANDARD.md`, `PLUGINS.md`, the files of the closest catalog plugin, and `benchcore/instrument.py`.

2 Paste the starter prompt

Use the one below, with your model filled in.

3 Probe

Ask for the read-only probe script. Read it, run it, paste the output back.

4 Design

Ask the measurement-design question from step 3 above. Insist on page numbers.

5 Write

Ask for the five files, one code block each, and save them into `catalog/[ID]/`.

6 Check

Run `python3 tools/check_plugin.py catalog/[ID]` and paste the whole output back. Repeat until PASS.

7 Test and finish

Ask for a test plan, run it, and paste the numbers back; the pass criteria are the same as above. Then have it fill in `tested_on` and the README from the results you pasted.

Long chats drift. If you come back to it later, re-upload the current manifest and driver rather than trusting it to remember them.

Starter prompt

For Claude.ai, or for Claude Code if you want to be explicit.

STARTER PROMPT

```
I'm writing a BenchPanel instrument plugin for the [MAKE MODEL]
(serial prefix [PREFIX]), connected by [GPIB via a LAN gateway / LAN /
USB].

Attached: its programming guide, PLUGIN_STANDARD.md, PLUGINS.md, the
[closest] catalog plugin as a model, and benchcore/instrument.py.

Rules:
1. Every SCPI command must come from the attached guide, with the page
   number. If the guide doesn't say, say so. Do not use other models'
manuals.
2. First confirm the guide's title page covers my model and serial prefix.
3. Nothing may block the bus: start a measurement, check completion with a
   register or event that never blocks, only then fetch. Handle "no
signal".
4. First give me a READ-ONLY probe script (queries only, no measurements),
   then wait for its output before designing anything.
5. Any action that can switch an output on must be declared "confirm" in
   the manifest and refuse without "confirmed": true.
6. Record anything on_connect changes and restore it in on_release.
7. The plugin must pass tools/check_plugin.py with no MUST failures.
8. Don't tell me it works; tell me what was measured and against what.

Start with rule 2, then the probe.
```

Reviewing what Claude wrote

Before it goes near the instrument, read these yourself. None take long.

LOOK AT	YOU WANT TO SEE
`driver.py` **docstring**	the manual's title and part number, and page numbers next to the commands
every `query(...)`	nothing that waits for a measurement unless it has already been told one finished
`manifest.json` **actions**	every action that can turn an output on has `"confirm"`
`all_off()`	the off command for every output, returning True only when that worked
`on_connect()` / `on_release()`	whatever the first changes, the second puts back
`has_output`	`true` if it can source power or signal, however small
`tested_on`	what was actually tested, when and how; not what it "should" work on
`quirks`	the surprises you found, written for the next person

Where Claude tends to go wrong

All of these happened while the catalog plugins were written.

WHAT HAPPENS	GUARD
Uses a sister model's manual because the file name looked right	Rule 2: check the title page
Cites a page from a different document	Ask it to verify each citation against this manual

WHAT HAPPENS	GUARD
Writes a blocking `READ?`: fine with a signal, a wedged bus without one	Settle the non-blocking design before any code
Checks the panel straight after an action and sees the old value (the panel updates on its next poll, about half a second later)	Wait for the next poll, or read the instrument directly
"All checks pass" from a check that tested nothing	Ask to see the check fail on a deliberately broken copy. That's what `tests/test_conformance.py` does, and its first run found a real gap in the checker
Reports success from a test that stopped early	Ask for the full output, not the summary
Trusts a web search summary of a spec	Download the actual document and read it

Sharing your plugin

- It must pass `python3 tools/check_plugin.py catalog/[ID]` with no MUST failures.
- `tested_on` must be honest. If you list a connection type you haven't tested, say so in the README.
- **No addresses in the plugin.** The checker refuses IP and GPIB addresses; they belong in each user's `site.json`.
- Zip the folder and share the zip. The recipient installs it with **Instruments → Check this .zip**, sees exactly what's in it, and confirms.